

Dinamik Programlama

Dinamik programlama, problemleri daha küçük alt problemlere ayırarak çözmeyi amaçlayan bir algoritma tasarım tekniğidir. Böl ve fethet yöntemine benzer, ancak en büyük farkı **alt problemlerin çakışması ve bu çözümlerin tekrar kullanılabilmesidir**.

Temel Kavramlar

1. Optimal Altyapı (Optimal Substructure)
 - Bir problemin optimal çözümü, alt problemlerinin optimal çözümlerini içerir.
 - Örneğin, En Uzun Ortak Alt Dizi (LCS) probleminde, eğer z LCS ise, z 'nin her öneki de iki dizinin önekleri arasında bir LCS olacaktır.
2. Alt Problemlerin Çakışması (Overlapping Subproblems)
 - Özyinelemeli çözüm yöntemi, az sayıda farklı alt problemi birçok kez tekrar çözer.
 - Eğer çözüm daha önce hesaplanıp saklanırsa, tekrar hesaplama yapılmadan doğrudan tabloya bakılarak sonuç elde edilir.

Örnek Problem: En Uzun Ortak Alt Dizi (LCS)

LCS problemi, iki dizgi arasında en uzun ortak alt diziyi bulmayı amaçlar.

Kaba Kuvvet (Brute Force) Yaklaşımı

- $x[1..m]$ ve $y[1..n]$ dizilerinin her alt dizisini oluşturup karşılaştırarak en uzun olanını seçmek.
- Ancak bu yöntem **üstel zaman karmaşıklığına** ($O(2^m)$) sahiptir ve büyük girdiler için uygulanamaz.

Daha İyi Bir Yaklaşım: Özyinelemeli Çözüm

- LCS uzunluğunu belirlemek için **özyinelemeli formül** kullanılır:
- $$c[i, j] = \begin{cases} c[i-1, j-1] + 1, & \text{eğer } x[i] = y[j] \text{ ise} \\ \max(c[i-1, j], c[i, j-1]), & \text{aksi takdirde} \end{cases}$$

Gördüğümüz (i, j) notasyonu, özyinelemeli çağrılar sırasında belirli bir alt problemi ifade eder. Bu notasyon genellikle dinamik programlamada matris tabanlı problemlerde kullanılır, özellikle En Uzun Ortak Alt Dizi (LCS) gibi problemlerde kullanılır. Bu yöntemde alt problemlerin tekrarlanması büyük bir performans kaybına neden olur.

Hatırlama (Memoization) Tekniği

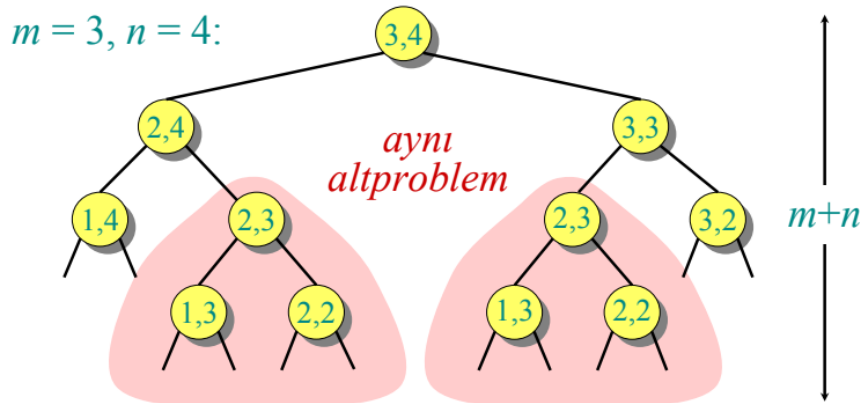
- Özyinelemeli yöntemin bir alt problemini çözdüğümüzde, sonucu **bir tabloda saklarız**.
- Böylece aynı alt problemle tekrar karşılaşıldığında hesaplamak yerine tabloya bakarak sonucu doğrudan alırız.
- Bu teknik **zaman karmaşıklığını** $O(mn)$ 'ye düşürür.

Dinamik Programlama ile LCS Hesaplama (Aşağıdan Yukarıya Yöntemi)

- Tablo (Matrix) Kullanımı
 - İlk olarak, dizilerin uzunluğuna bağlı olarak bir **tablo (matris)** oluşturulur.
 - **Tablonun her hücresi**, o noktaya kadar bulunan en uzun ortak alt dizinin uzunluğunu temsil eder.
 - Tablodaki en sağ alt köşe, LCS'in uzunluğunu verir.
 - **Geriyeye dönerek** LCS'nin kendisi de elde edilebilir.
- Zaman Karmaşıklığı: $O(mn)$
- **Alan (Hafıza) Karmaşıklığı:** $O(mn)O(mn)$, ancak bazen $O(\min(m,n))$ olarak optimize edilebilir.

Sonuç olarak, dinamik programlama, büyük problemleri küçük parçalara ayırarak ve tekrar eden hesaplamaları önleyerek **verimli algoritmalar** geliştirilmesini sağlar. En Uzun Ortak Alt Dizi (LCS) gibi problemler, **kaba kuvvet ve özyinelemeli yöntemlerden çok daha hızlı çözülebilir**.

Özyineleme Ağacındaki Dğümler Ne Anlama Geliyor?



- Yükseklik = $m + n \Rightarrow$ iş potansiyel olarak üsteldir,
1. ama biz zaten çözülmüş olan altproblemleri çözüyoruz!

Şekildeki (i, j) çiftleri, verilen iki dizinin **ilk i elemanı** ile **ilk j elemanını** kullanarak problemi çözmeye çalıştığımız alt problemleri gösterir.

Bu, genellikle **En Uzun Ortak Alt Dizi (LCS) problemi** için aşağıdaki şekilde yorumlanır:

- $(i, j) \rightarrow$ x dizisinin ilk i karakteri ile y dizisinin ilk j karakteri arasındaki en uzun ortak alt diziyi bulmaya çalışıyoruz.
- Özyineleme sırasında büyük problem $(3,4)$ çözülmeye başlanır ve daha küçük alt problemlere bölünerek ilerlenir.

Örnek Açıklamalar

1. (3,4)
 - **x dizisinin ilk 3 karakteri ile y dizisinin ilk 4 karakteri** arasındaki en uzun ortak alt diziye bulmaya çalışıyoruz.
 - Bu alt problem çözülmeden önce, daha küçük alt problemlere ayrılacak.
2. (2,4)
 - x'in ilk 2 karakteri ile y'in ilk 4 karakteri arasındaki problemi çözüyoruz.
 - Bu, (3,4) düğümünün bir alt problemi.
3. (1,4)
 - x'in ilk 1 karakteri ile y'in ilk 4 karakteri arasındaki LCS hesaplanıyor.
 - Daha küçük bir alt problem olduğu için bu ağaçtaki yaprak düğümlerden biri.
4. (2,3)
 - x'in ilk 2 karakteri ile y'in ilk 3 karakteri arasındaki LCS bulunuyor.
 - (2,4) alt probleminden türetilmiş bir düğüm.

Neden Bu Yapı Kullanılıyor?

- Özyineleme işlemi büyük bir problemi küçültmeye dayanır.
- Özyineleme ağacı, problemin nasıl adım adım küçültüldüğünü gösterir.
- Bazı alt problemler birden çok kez hesaplanıyor (örneğin (2,3) düğümü iki farklı yoldan türetiliyor).
 - Bu gereksiz tekrarları önlemek için "dinamik programlama" devreye girer.
 - Daha önce hesaplanan alt problemlerin sonuçları saklanır (Memoization), böylece aynı problem tekrar hesaplanmaz.

Sonuç olarak (i, j) notasyonu, problemin alt problemlere bölünmesini ve dinamik programlamanın nasıl çalıştığını anlamak için önemlidir.

Özyinelemeli yaklaşım **üstel zaman karmaşıklığına** sahipken, **dinamik programlama (tablo yöntemi veya memoization)** kullanarak **O(mn)** zaman karmaşıklığında bir çözüm elde edilebilir.

Needleman-Wunsch Algoritması

Needleman-Wunsch algoritması, biyoinformatikte **DNA, RNA veya protein dizilerini hizalamak** için kullanılan **küresel hizalama** algoritmalarından biridir. Dinamik programlama temelli olan bu algoritma, iki diziyi tamamen eşleştirerek **optimal hizalamayı** bulmayı amaçlar.

Algoritma, iki diziyi hizalarken **eşleşme (match)**, **uyumsuzluk (mismatch)** ve **boşluk (gap)** cezaları kullanarak en iyi hizalamayı bulur.

Diziler aşağıdaki gibi olabilir:

X = G A T T A C A

Y = G C A T G C

Bu diziler arasında mümkün olan en iyi hizalamayı bulmak için **dinamik programlama** kullanılır.

Adımlar

1. Matris (Tablo) Oluşturma

- $F[i,j]$ matrisi oluşturulur.
- Satırlar birinci dizinin karakterlerini, sütunlar ikinci dizinin karakterlerini temsil eder.
- İlk satır ve ilk sütun, boşluk (gap) cezalarına göre doldurulur.

2. Matrisin Doldurulması

- Her hücre üç farklı yoldan biriyle doldurulur:
 1. **Diagonal hareket** (eşleşme veya uyumsuzluk)
 2. **Yukarı hareket** (boşluk ekleme)
 3. **Sola hareket** (boşluk ekleme)
- Formül:

$$F[i,j] = \max \begin{cases} F[i-1,j-1] + \text{skor}(X_i, Y_j) & \text{(Eşleşme veya uyumsuzluk)} \\ F[i-1,j] + \text{gap cezası} & \text{(Silme)} \\ F[i,j-1] + \text{gap cezası} & \text{(Ekleme)} \end{cases}$$

3. Geri İzleme (Backtracking)

- Matris tamamlandıktan sonra, **geri izleme** ile en iyi hizalama bulunur.
- Bu işlem, en sağ alt köşeden başlayarak **maksimum skorlu yollardan** geri giderek hizalamayı oluşturur.

Örnek

Parametreler

- Eşleşme skoru (Match Score) = +1
- Uyumsuzluk skoru (Mismatch Score) = -1
- Boşluk (Gap) cezası = -2

Diziler

X = G A T T A C A

Y = G C A T G C

Matris Doldurma

$$F[i, j] = \max \begin{cases} F[i-1, j-1] + \text{skor}(X_i, Y_j) & \text{(Eşleşme veya uyumsuzluk)} \\ F[i-1, j] + \text{gap cezası} & \text{(Silme)} \\ F[i, j-1] + \text{gap cezası} & \text{(Ekleme)} \end{cases}$$

Örneğin, başlangıç matrisinin ilk satırı ve ilk sütunu boşluk cezalarına göre doldurulur. Ardından, yukarıdaki formül kullanılarak diğer hücreler doldurulur.

Online çözüm yapan Needleman Wunsch algoritmasına

<https://rna.informatik.uni-freiburg.de/Teaching/index.jsp?toolName=Needleman-Wunsch>

linkinden ulaşabilirsiniz.

Input:

Sequence *a*:

Sequence *b*:

Optimization of: Distance ☐ Similarity ☒

Scoring in *s*: Match Mismatch Gap

Hint:
For similarity maximization,
match scores should be positive and all other scores lower.
For distance minimization the reverse applies.

Recursion:
$$D_{i,j} = \max \begin{cases} D_{i-1,j-1} + s(a_i, b_j) \\ D_{i-1,j} + s(a_i, -) \\ D_{i,j-1} + s(-, b_j) \end{cases} = \max \begin{cases} D_{i-1,j-1} + 1 & a_i = b_j \\ D_{i-1,j-1} + -1 & a_i \neq b_j \\ D_{i-1,j} + -2 & b_j = - \\ D_{i,j-1} + -2 & a_i = - \end{cases}$$

Output:

D		G ₁	C ₂	A ₃	T ₄	G ₅	C ₆
	0	-2	-4	-6	-8	-10	-12
G ₁	-2	1	-1	-3	-5	-7	-9
A ₂	-4	-1	0	0	-2	-4	-6
T ₃	-6	-3	-2	-1	1	-1	-3
T ₄	-8	-5	-4	-3	0	0	-2
A ₅	-10	-7	-6	-3	-2	-1	-1
C ₆	-12	-9	-6	-5	-4	-3	0
A ₇	-14	-11	-8	-5	-6	-5	-2
Score: -2							

Results

You can select a result to get the related traceback.

GATTACA

* | | * | *

GCATGC_

Geri İzleme (Backtracking)

Geri izleme ile en iyi hizalamayı oluştururuz:

X	G	A	T	T	A	C	A
	*			*		*	
Y	G	C	A	T	G	C	-

Karmaşıklık Analizi

- **Zaman Karmaşıklığı:** $O(mn)$, burada m ve n iki dizinin uzunluklarıdır.
- **Alan (Hafıza) Kullanımı:** $O(mn)$ (optimize edilirse $O(\min(m, n))$)

Özetle

- Needleman-Wunsch, **küresel hizalama** algoritmasıdır.
- **Dinamik programlama** kullanarak en iyi hizalamayı bulur.
- Boşluk cezaları, eşleşmeler ve uyumsuzluklar hesaba katılır.
- Biyoinformatikte DNA, RNA ve protein dizileri için yaygın olarak kullanılır.

Needleman-Wunsch algoritması, Smith-Waterman algoritmasının aksine küresel hizalama yaptığı için dizilerin tamamını kapsayan hizalamalar üretir. Eğer yerel hizalama yapmak istersen, Smith-Waterman algoritmasını kullanmak gerekir.

Smith Waterman Algoritması

Online çözüm yapan Needleman Wunsch algoritmasına

<https://rna.informatik.uni-freiburg.de/Teaching/index.jsp?toolName=Smith-Waterman>

linkinden ulaşabilirsiniz.

Parametreler

- Eşleşme skoru (Match Score) = +1
- Uyumsuzluk skoru (Mismatch Score) = -1
- Boşluk (Gap) cezası = -2

Diziler

- X = G A T T A C A
- Y = G C A T G C

Input:

Sequence *a*:

Sequence *b*:

Scoring in *s*: Match Mismatch Gap

Hint:
For similarity maximization,
match scores should be positive and all other scores lower.

Recursion:
$$S_{i,j} = \max \begin{cases} S_{i-1,j-1} + s(a_i, b_j) \\ S_{i-1,j} + s(a_i, -) \\ S_{i,j-1} + s(-, b_j) \\ 0 \end{cases} = \max \begin{cases} S_{i-1,j-1} + 1 & a_i = b_j \\ S_{i-1,j-1} + -1 & a_i \neq b_j \\ S_{i-1,j} + -2 & b_j = - \\ S_{i,j-1} + -2 & a_i = - \\ 0 \end{cases}$$

Output:

S		G_1	C_2	A_3	T_4	G_5	C_6
	0	0	0	0	0	0	0
G_1	0	1	0	0	0	1	0
A_2	0	0	0	1	0	0	0
T_3	0	0	0	0	2	0	0
T_4	0	0	0	0	1	1	0
A_5	0	0	0	1	0	0	0
C_6	0	0	1	0	0	0	1
A_7	0	0	0	2	0	0	0
Score: 2							

Results

You can select a result to get the related traceback.

AT
**
AT
CA
**
CA

Output:

S		G_1	C_2	A_3	T_4	G_5	C_6
	0	0	0	0	0	0	0
G_1	0	1	0	0	0	1	0
A_2	0	0	0	1	0	0	0
T_3	0	0	0	0	2	0	0
T_4	0	0	0	0	1	1	0
A_5	0	0	0	1	0	0	0
C_6	0	0	1	0	0	0	1
A_7	0	0	0	2	0	0	0
Score: 2							

Results

You can select a result to get the related traceback.

AT
**
AT
CA
**
CA